

1. Title

Digital Certificate Forgery Detection Using Image Registration, XOR Difference Analysis and Morphological Processing

2. Aim

To develop a software-based certificate validation system capable of detecting tampering in digital or scanned academic certificates using classical image processing techniques by identifying forged textual modifications, localizing suspicious regions, and generating an authenticity score.

3. Software Tools Used

- Python 3.11
 - OpenCV (cv2)
 - NumPy
 - Visual Studio Code (VS Code)
-

4. Approaches Implemented and Problems Faced

Approach 1: Basic Pixel Difference Method

Initially, direct subtraction between original and test certificate images was performed followed by thresholding and contour detection to identify altered regions.

Problems Faced:

- Highly sensitive to scan noise and minor alignment shifts
- Font anti-aliasing caused false differences
- Large number of unchanged regions were wrongly flagged as tampered

Outcome:

This method was unreliable for practical certificate validation.

Approach 2: Registration-Assisted Structural Comparison

To reduce false positives, preprocessing and ORB feature-based homography alignment were introduced before comparison.

Problems Faced:

- Alignment improved matching but minor rendering variations still produced artifacts

- Text-edge irregularities triggered false tampering regions

Outcome:

Misalignment effects were reduced, but the method still lacked robustness.

Approach 3: Edge and Block-Based Difference Analysis

Additional experiments were performed using:

- Edge anomaly detection
- Block-wise difference grids
- Local structural comparisons

Problems Faced:

- Signature and border regions dominated edge responses
- Block boundaries caused poor localization
- Precision remained low

Outcome:

These methods were discarded due to instability.

Approach 4: Binary XOR with Morphological Filtering (Final Core Approach)

A more reliable method was developed using:

- Binary thresholding
- XOR difference detection
- Morphological opening
- Dilation
- Contour filtering

Problems Faced:

- Small noise blobs initially caused false detections
- Underline artifacts interfered with contour detection
- Threshold tuning required balancing missed edits and over-detection

Outcome:

This became the final stable tamper localization approach.

Approach 5: Authenticity Scoring Framework

A heuristic authenticity score was introduced to quantify tampering severity.

Outcome:

Provided numerical confidence in addition to visual tamper marking.

Major Challenges Encountered

1. Registration drift causing false differences
2. Anti-aliasing sensitivity in unchanged text
3. Noise-induced contour over-detection
4. Precision vs sensitivity threshold tradeoff

Overall Development Summary

The project evolved from naïve pixel comparison to a registration-assisted morphological document forgery detection system through iterative experimentation, debugging and refinement, reflecting systematic engineering development.

5. Algorithms / Methodology Used

Step 1: Document Preprocessing

Each certificate image was normalized using:

- Image resizing (1200×1600)
- Grayscale conversion
- CLAHE contrast enhancement
- Gaussian filtering for noise reduction
- Sharpening filter for text edge enhancement

Purpose: Improve consistency between reference and test documents.

Step 2: Image Registration and Alignment

To eliminate scan misalignment effects:

- ORB feature extraction was applied
- Feature matching performed using Brute Force Matcher
- Homography matrix estimated using RANSAC

- Perspective warping aligned the test certificate with the original template

Purpose: Reduce false tamper detection caused by geometric shifts.

Step 3: Binary XOR Difference Detection

After alignment:

- Binary thresholding converted text regions into foreground masks
- Bitwise XOR operation detected differences between original and test certificates

This highlighted altered regions.

Step 4: Morphological Noise Filtering

To suppress false detections:

- Morphological opening removed isolated noise
- Dilation improved change visibility
- Contour filtering rejected:
 - Very small regions (noise)
 - Large blobs (artifacts)
 - Horizontal underline-type artifacts

This improved localization accuracy.

Step 5: Authenticity Scoring

Authenticity score was calculated using:

$$Score = 100 - \left(\frac{Changed\ Pixels}{Total\ Pixels} \times 100 \times 6 \right)$$

Classification criteria:

- Score > 90 → Authentic
 - Score 70–90 → Minor Tampering
 - Score < 70 → High Tampering
-

6. Results

The system was tested using genuine and forged certificates.

Test Image	Authenticity Score	Result
Genuine Certificate	100.00%	Authentic
Date, GR Edited Certificate	40.46%	High Tampering
Heavy Forged Certificate	29.35%	High Tampering
Multi-field Forged Certificate	51.57%	High Tampering

Observations

- Genuine certificate produced no false tamper detection and was correctly classified as authentic.
- Date-edited forgery was detected with localized tampering boxes.
- Heavy forged certificate showed widespread anomalies and lowest authenticity score.
- Multi-field forgery was successfully detected and classified as highly tampered.

Achievements

- Successfully built software-only certificate forgery detector
- Implemented image registration assisted tamper detection
- Achieved localized tamper marking with bounding boxes
- Reduced false positives through morphology and contour filtering
- Developed quantitative authenticity scoring framework

7. Conclusion

A certificate forgery detection system based on image registration, XOR differencing and morphological analysis was successfully implemented.

The system is capable of:

- Detecting localized certificate modifications
- Highlighting suspicious regions visually
- Distinguishing genuine and forged certificates
- Assigning quantitative authenticity scores

Experimental results demonstrated successful detection of date edits, multi-field alterations and heavily forged certificates while correctly validating genuine certificates.

The implemented system provides a practical foundation for software-based document forensic verification.

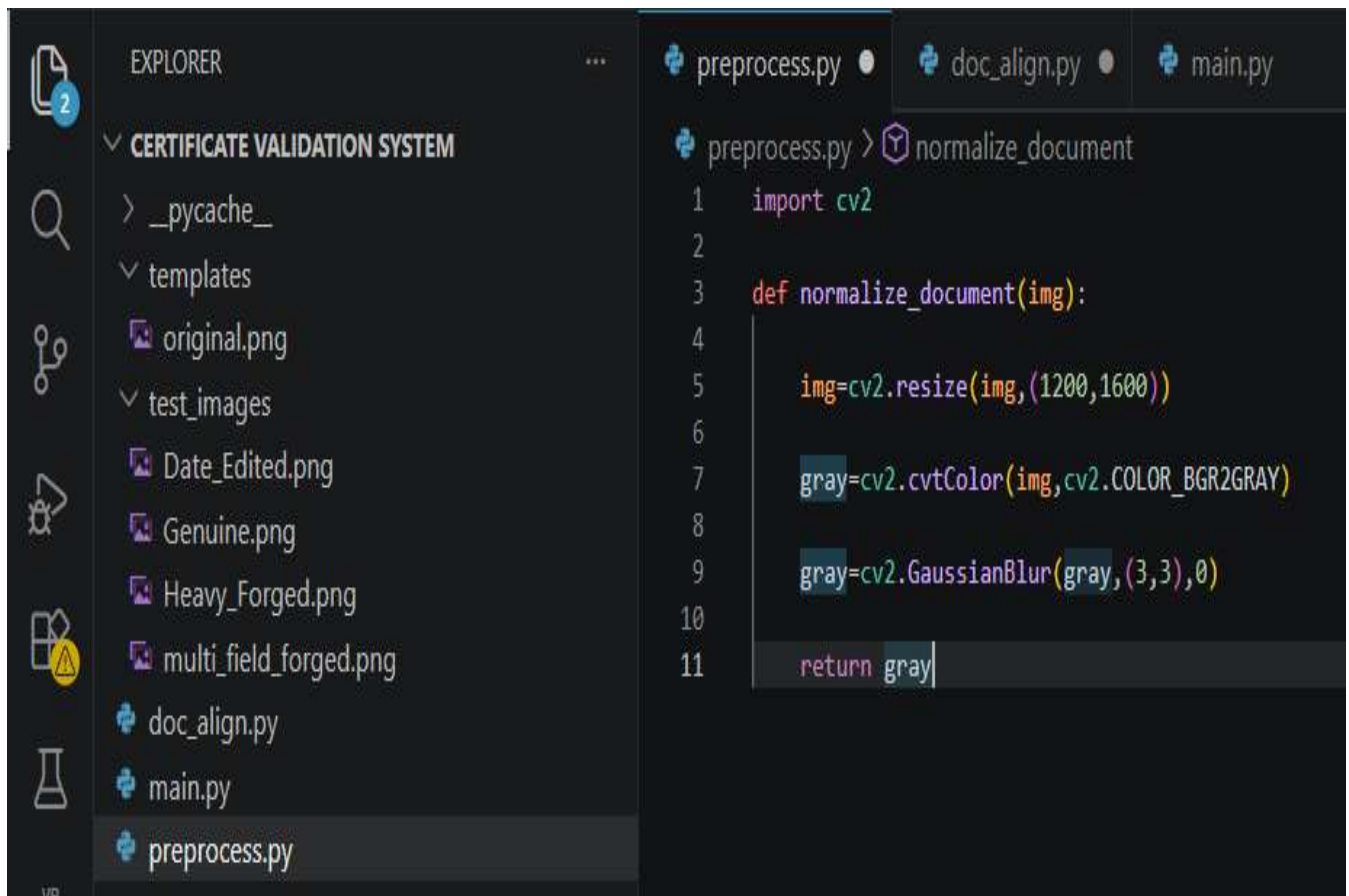
8. Future Scope

Future improvements may include:

- OCR-assisted semantic validation
- Field-level intelligent verification (Name, GR No., Date)
- GUI or web-based deployment
- Confidence heatmap visualization
- Multi-template certificate validation support

9. Source Code

Code 1: Preprocessing Module



The screenshot displays a code editor interface with a dark theme. On the left, the 'EXPLORER' sidebar shows a project structure for 'CERTIFICATE VALIDATION SYSTEM'. It includes a '__pycache__' directory, a 'templates' folder, and several image files: 'original.png', 'Date_Edited.png', 'Genuine.png', 'Heavy_Forged.png', and 'multi_field_forged.png'. There are also three Python files: 'doc_align.py', 'main.py', and 'preprocess.py', which is currently selected. The main editor area shows the 'preprocess.py' file with the 'normalize_document' function. The function takes an 'img' parameter and performs three operations: resizing the image to (1200, 1600), converting it to grayscale using 'cv2.cvtColor', and applying Gaussian blur with a kernel size of (3, 3) and sigma of 0. The function returns the processed 'gray' image.

```
preprocess.py | doc_align.py | main.py
preprocess.py > normalize_document
1  import cv2
2
3  def normalize_document(img):
4
5      img=cv2.resize(img,(1200,1600))
6
7      gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
8
9      gray=cv2.GaussianBlur(gray,(3,3),0)
10
11  return gray
```

Code 2: Document Alignment Module

```
doc_align.py • preprocess.py •
doc_align.py > register_images
1 import cv2
2 import numpy as np
3 def align_document(path):
4
5     img=cv2.imread(path)
6
7     img=cv2.resize(
8         img,
9         (1200,1600)
10    )
11    return img
12 def register_images(reference,test):
13
14     gray1=cv2.cvtColor(
15         reference,
16         cv2.COLOR_BGR2GRAY
17     )
18
19     gray2=cv2.cvtColor(
20         test,
21         cv2.COLOR_BGR2GRAY
22     )
23     orb=cv2.ORB_create(5000)
24
25     kp1,des1=orb.detectAndCompute(
26         gray1,None
27     )
28     kp2,des2=orb.detectAndCompute(
29         gray2,None
30     )
31     matcher=cv2.BFMatcher(
32         cv2.NORM_HAMMING,
33         crossCheck=True
34     )
35     matches=matcher.match(
36         des1,
37         des2
38     )
39     matches=sorted(
40         matches,
41         key=lambda x:x.distance
42    )[:500]
43     pts1=np.float32(
44         [kp1[m.queryIdx].pt for m in matches]
45     ).reshape(-1,1,2)
46     pts2=np.float32(
47         [kp2[m.trainIdx].pt for m in matches]
48     ).reshape(-1,1,2)
49     H,_=cv2.findHomography(
50         pts2,
51         pts1,
52         cv2.RANSAC
53     )
54     aligned=cv2.warpPerspective(
55         test,
56         H,
57         (
58             reference.shape[1],
59             reference.shape[0]
60         )
61     )
62     return aligned
```

Code 3: Main Forgery Detection Program

```
main.py preprocess.py doc_align.py

main.py > ...
1  import cv2
2  import os
3  import numpy as np
4
5  from doc_align import align_document, register_images
6  original=align_document(
7  "templates/original.png"
8  )
9  for file in os.listdir("test_images"):
10
11      print("\nTesting:", file)
12
13      test=align_document(
14          | f"test_images/{file}"
15      )
16
17      test=register_images(
18          | original,
19          | test
20      )
21      g1=cv2.cvtColor(
22          | original,
23          | cv2.COLOR_BGR2GRAY
24      )
25      g2=cv2.cvtColor(
26          | test,
27          | cv2.COLOR_BGR2GRAY
28      )
29      output=test.copy()
30
31      g1[0:250,:]=255
32      g2[0:250,:]=255
33      _,b1=cv2.threshold(
34          | g1,180,255,
35          | cv2.THRESH_BINARY_INV
36      )
37      _,b2=cv2.threshold(
38          | g2,180,255,
39          | cv2.THRESH_BINARY_INV
40      )
41      diff=cv2.bitwise_xor(
42          | b1,b2
43      )
44      kernel=np.ones(
45          | (2,2),
46          | np.uint8
47      )
48      diff=cv2.morphologyEx(
49          | diff,
50          | cv2.MORPH_OPEN,
51          | kernel
52      )
53      diff=cv2.dilate(
54          | diff,
55          | kernel,
56          | iterations=1
57      )
58      changed=cv2.countNonZero(diff)
59      total=diff.shape[0]*diff.shape[1]
60      tamper=(changed/total)*100
61      tamper*=6
62      if tamper>100:
63          | tamper=100
64      score=100-tamper
```

main.py

preprocess.py

main.py > ...

```
57     )
58     changed=cv2.countNonZero(diff)
59     total=diff.shape[0]*diff.shape[1]
60     tamper=(changed/total)*100
61     tamper*=6
62     if tamper>100:
63         tamper=100
64     score=100-tamper
65     if score>90:
66         status="AUTHENTIC"
67
68     elif score>70:
69         status="MINOR TAMPERING"
70
71     else:
72         status="HIGH TAMPERING"
73     print(
74         f"Authenticity Score: {score:.2f}%"
75     )
76     print(
77         "Status:",
78         status
79     )
80     contours, _=cv2.findContours(
81         diff,
82         cv2.RETR_EXTERNAL,
83         cv2.CHAIN_APPROX_SIMPLE
84     )
85     for c in contours:
86
87         x,y,w,h=cv2.boundingRect(c)
88         area=w*h
89
90         if area<120:
91             continue
92
93         if area>2500:
94             continue
95
96         if w>4*h:
97             continue
98
99         cv2.rectangle(
100             output,
101             (x,y),
102             (x+w,y+h),
103             (0,0,255),
104             2
105         )
106     cv2.putText(
107         output,
108         f"{score:.1f}%",
109         (40,80),
110         cv2.FONT_HERSHEY_SIMPLEX,
111         1,
112         (0,0,255),
113         3
114     )
115     cv2.imshow(
116         file,
117         output
118     )
119     cv2.waitKey(0)
120     cv2.destroyAllWindows()
```

10. Output

Output 1 — Genuine Certificate Detection

Output 2 — Multi-field Forgery Detection

Output 3 — Heavy Forgery Detection

Output 4 — Date, GR Forgery Detection

Console Output

Testing: Date and GR Edited
Authenticity Score: 40.46%
Status: HIGH TAMPERING

Testing: Genuine
Authenticity Score: 100.00%
Status: AUTHENTIC

Testing: Heavy Forged
Authenticity Score: 29.35%
Status: HIGH TAMPERING

Testing: multi field
Authenticity Score: 51.57%
Status: HIGH TAMPERING